Name: Abhishek Kamlesh Vishwakarma
Class: D20C **Roll.No:** 35

Experiment No.5

Aim: Hands on Solidity Programming Assignments for creating Smart Contracts
Lab Objectives: To explore Blockchain concepts.
Lab Outcomes (LO): Design Smart Contract using Solidity (LO2)

Theory:

1. **Primitive Data Types, Variables, Functions – pure, view**

In Solidity, primitive data types form the foundation of smart contract development. Commonly used types include:

- **uint / int:** unsigned and signed integers of different sizes (e.g., uint256, int128).
- **bool:** represents logical values (true or false).
- **address:** holds a 20-byte Ethereum account address, often used for storing user accounts or contract addresses.
- **bytes / string:** store binary data or textual data.

Variables in Solidity can be **state variables** (stored on the blockchain permanently), **local variables** (temporary, created during function execution), or **global variables** (special predefined variables such as msg.sender, msg.value, and block.timestamp).

Functions allow execution of contract logic. Special types of functions include:

- **pure:** cannot read or modify blockchain state; they work only with inputs and internal computations.
- **view:** can read state variables but cannot alter them. This classification helps optimize gas usage and enforces function integrity.

2. **Inputs and Outputs to Functions**

Functions in Solidity can accept input arguments and return one or more output values. Inputs enable users or other contracts to pass data into the contract, while outputs make it possible to return results after computation. For example, a function can accept an amount in Ether and return whether the transfer was successful. Solidity also allows named return variables, which improve readability and debugging.

3. **Visibility, Modifiers and Constructors**

- **Function Visibility** defines who can access a function:
    - public: available both inside and outside the contract.
    - private: only accessible within the same contract.

    - internal: accessible within the contract and its child contracts.
    - external: can be called only by external accounts or other contracts.

- **Modifiers** are reusable code blocks that change the behavior of functions. They are often used for access control, such as restricting sensitive functions to the contract owner (onlyOwner).
- **Constructors** are special functions executed only once during contract deployment. They initialize important values, such as setting the deploying account as the owner of the contract.

4. **Control Flow: if-else, loops**

Control flow in Solidity is similar to traditional programming languages:

- **if-else** allows conditional decision-making in contract logic, e.g., checking if a balance is sufficient before transferring funds.
- **Loops** (for, while, do-while) enable repeated execution of code. For example, iterating through an array of users. However, loops must be used carefully, as excessive iterations increase gas consumption, potentially making the contract expensive to execute.

5. **Data Structures: Arrays, Mappings, Structs, Enums**

- **Arrays:** Can be fixed or dynamic and are used to store ordered lists of elements. Example: an array of addresses for registered users.
- **Mappings:** Key-value pairs that allow quick lookups. Example: mapping(address => uint) for storing balances. Unlike arrays, mappings do not support iteration.
- **Structs:** Allow grouping of related properties into a single data type, such as creating a struct Player {string name; uint score;}.
- **Enums:** Used to define a set of predefined constants, making code more readable. Example: enum Status { Pending, Active, Closed }.

6. **Data Locations**

Solidity uses three primary data locations for storing variables:

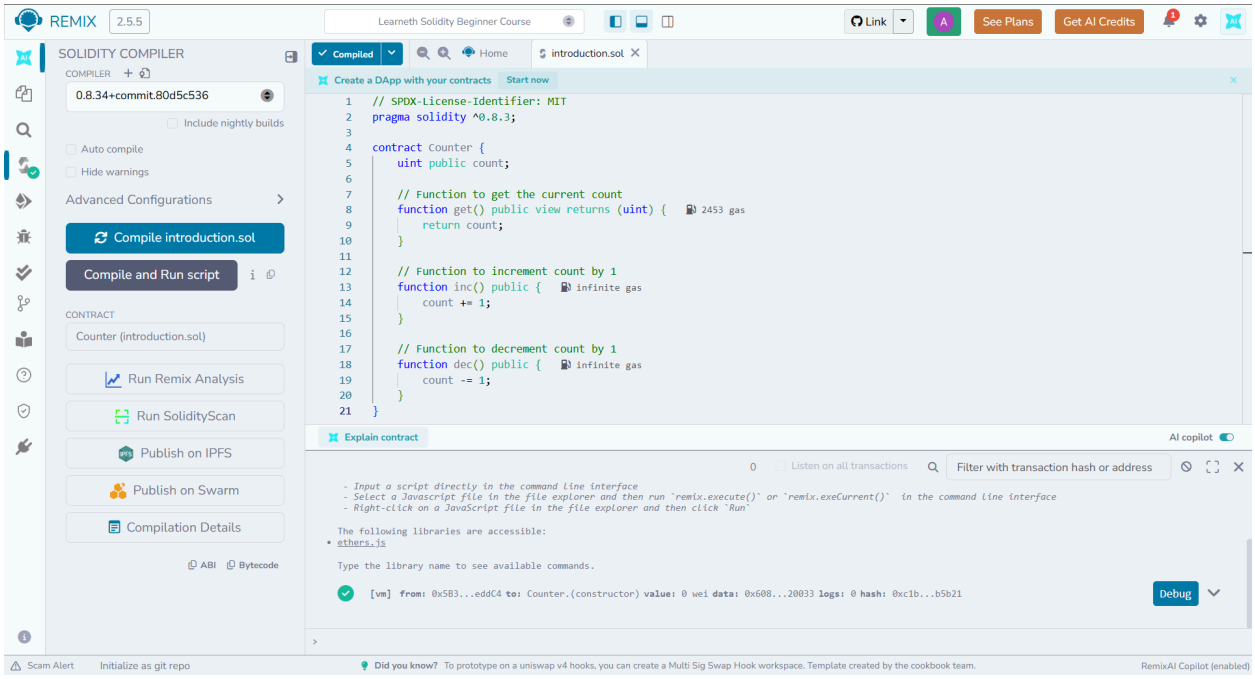
- **storage**: Data stored permanently on the blockchain. Examples: state variables.
- **memory**: Temporary data storage that exists only while a function is executing. Used for local variables and function inputs.
- **calldata**: A non-modifiable and non-persistent location used for external function parameters. It is gas-efficient compared to memory. Understanding data locations is essential, as they directly impact gas costs and performance.

7. **Transactions: Ether and Wei, Gas and Gas Price, Sending Transactions**

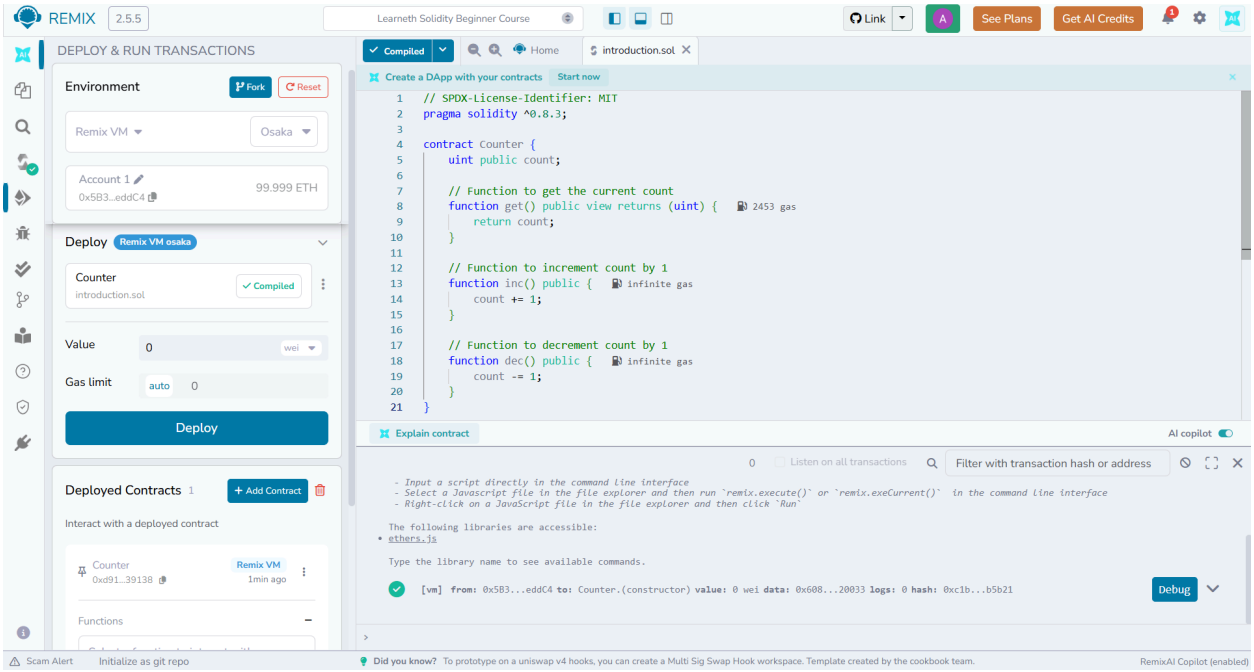
- **Ether and Wei**: Ether is the main currency in Ethereum. All values are measured in Wei, the smallest unit (1 Ether = 10^{18} Wei). This ensures high precision in financial transactions.
- **Gas and Gas Price**: Every transaction consumes gas, which represents computational effort. The gas price determines how much Ether is paid per unit of gas. A higher gas price incentivizes miners to prioritize the transaction.
- **Sending Transactions**: Transactions are used for transferring Ether or interacting with contracts. Functions like transfer() and send() are commonly used, while call() provides more flexibility. Each transaction requires gas, making efficiency in contract design very important.

Implementation:

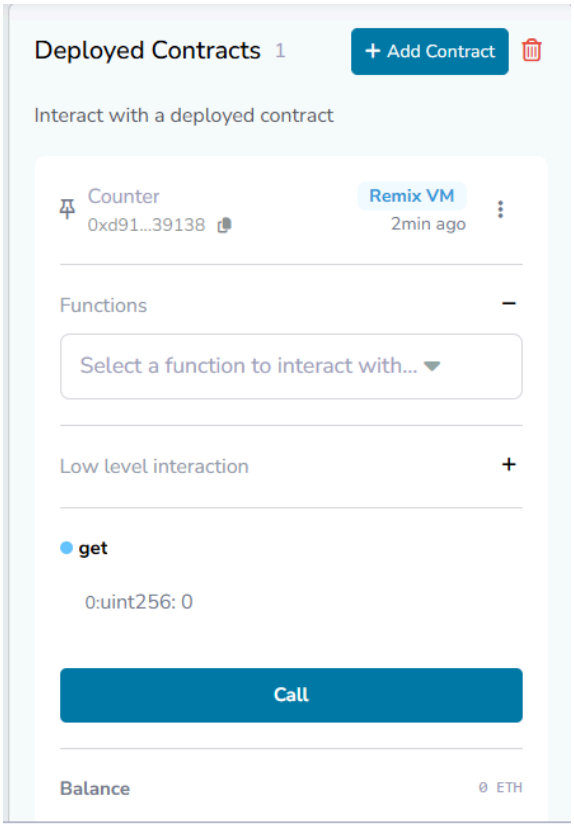
Tutorial 1: Compile the code



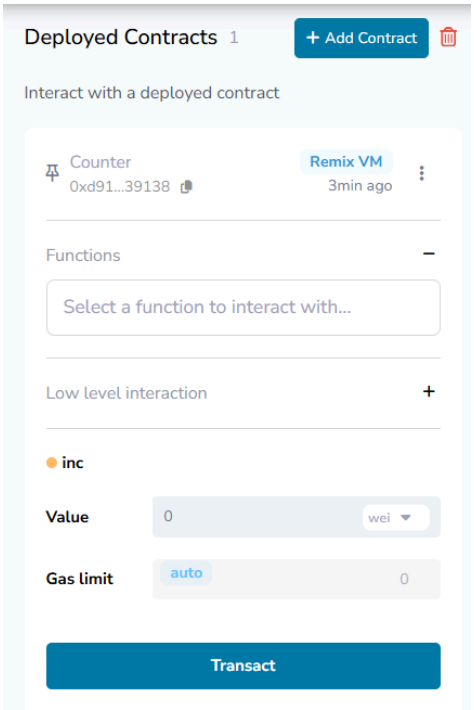
Tutorial 1: Deploy the contract



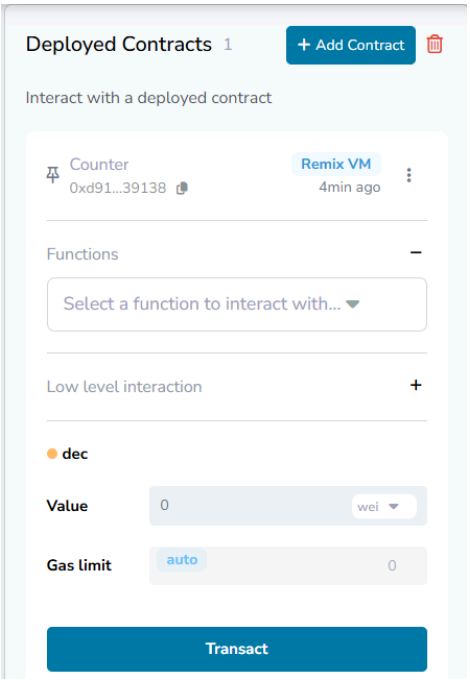
Tutorial 1: get

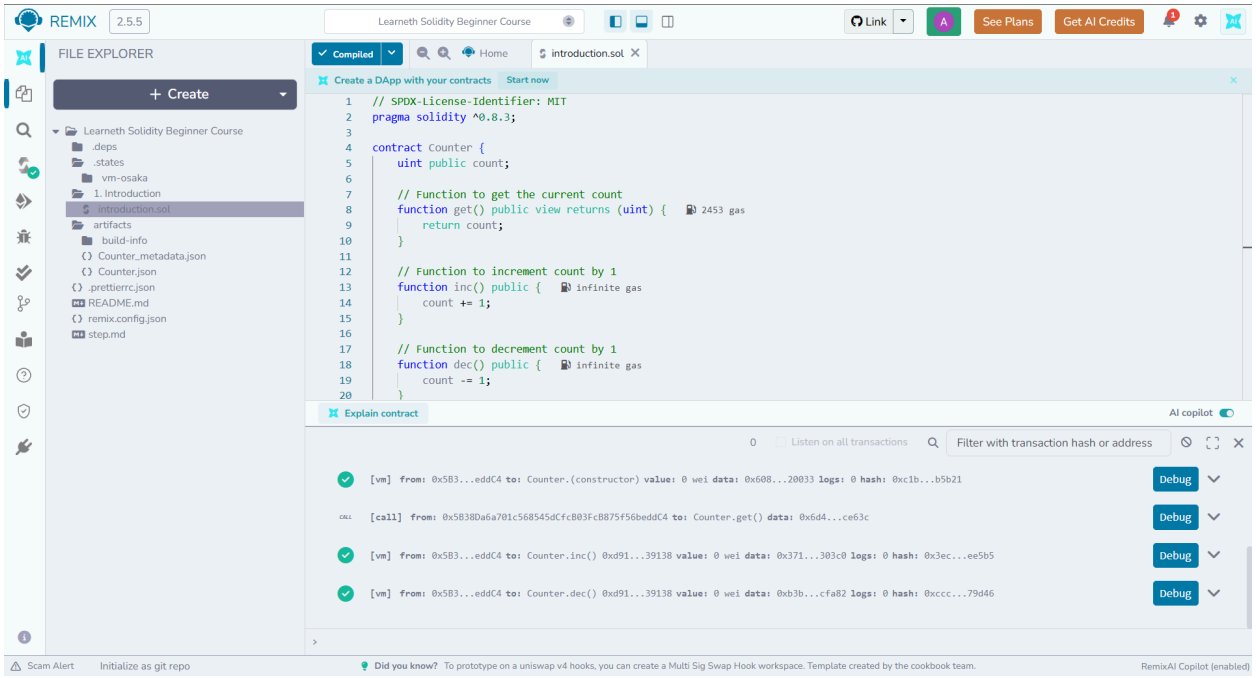


Tutorial 1: Increment

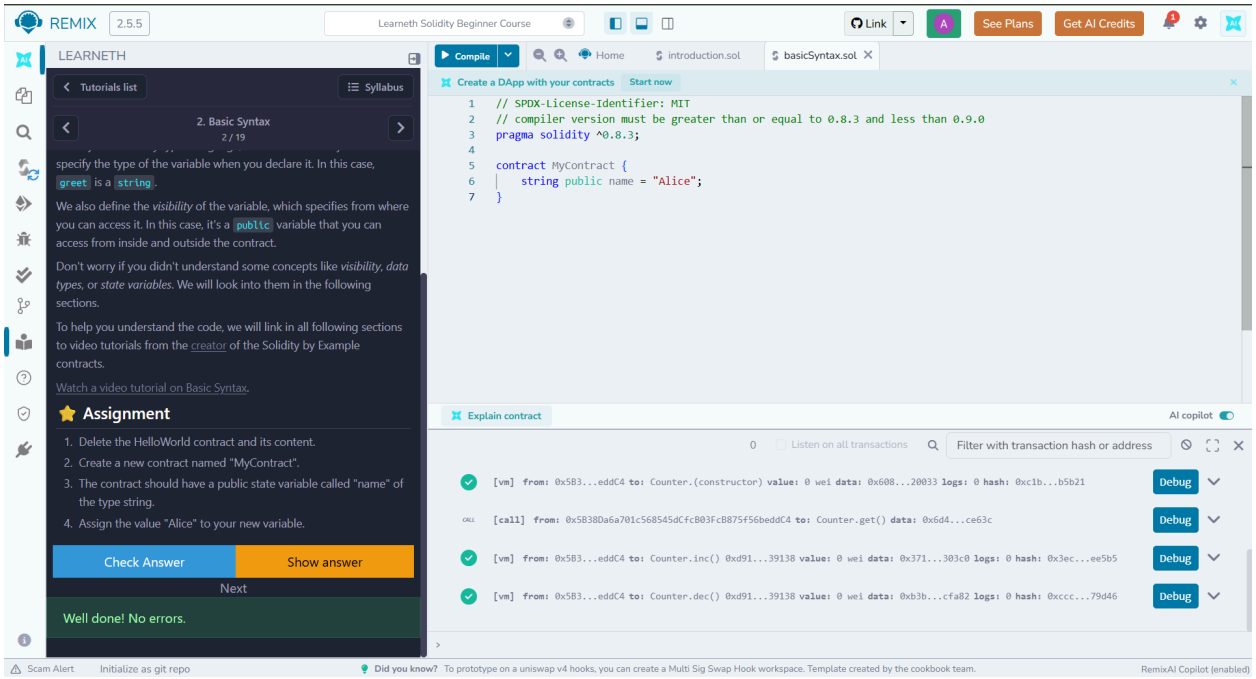


Tutorial 1: Decrement

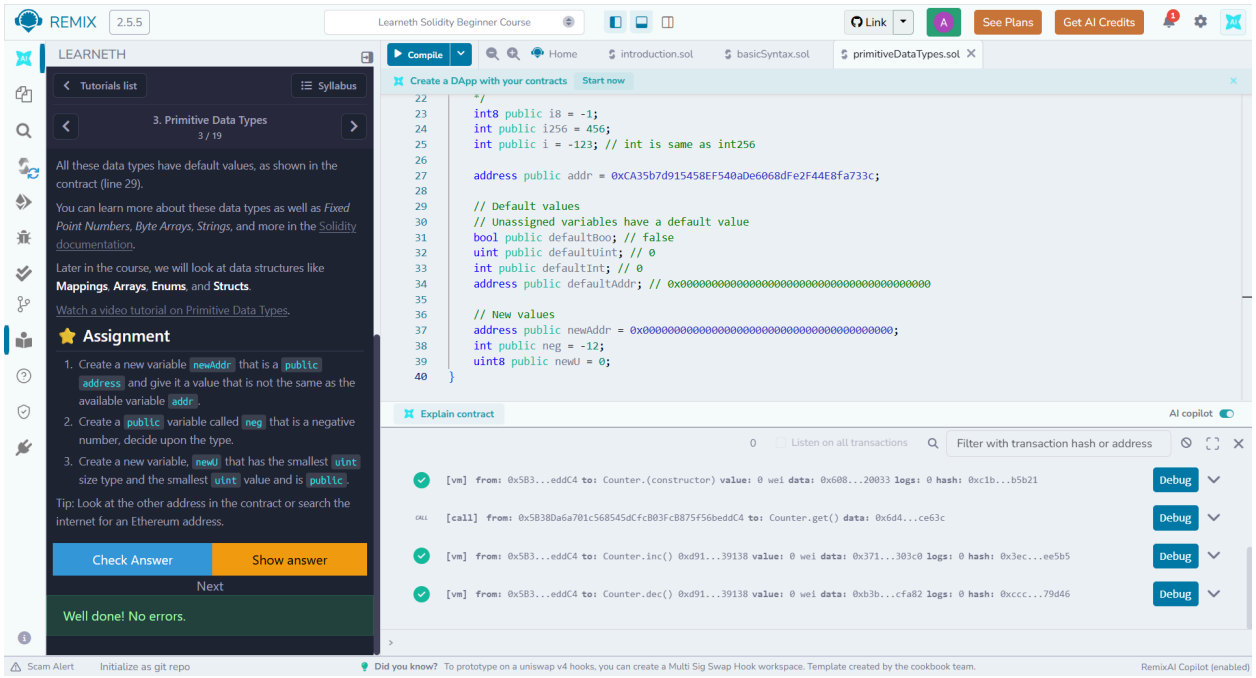




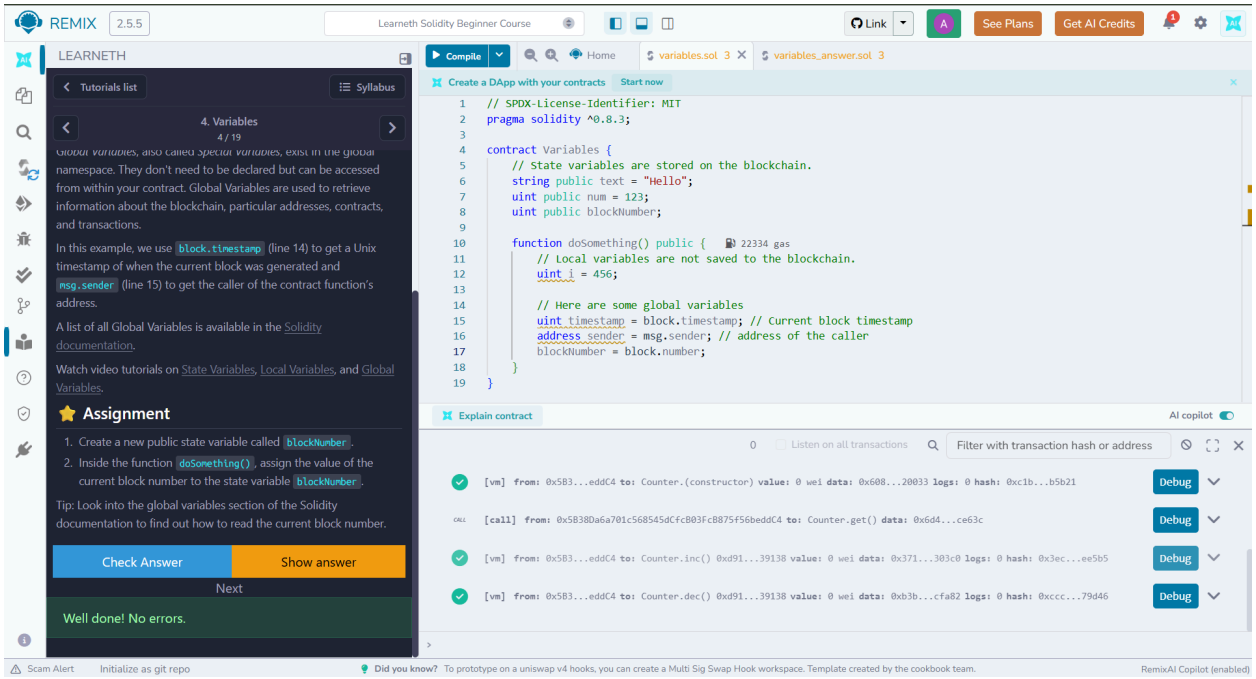
Tutorial 2



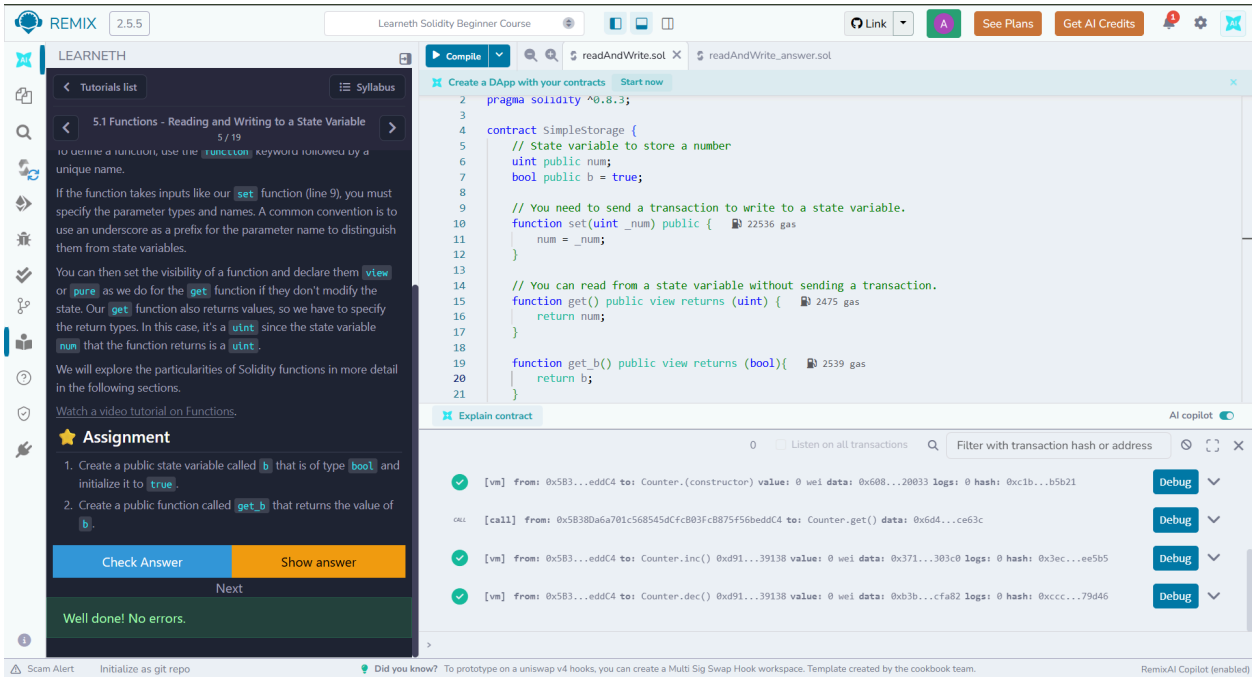
Tutorial 3


REMIX 2.5.5
Learneth Solidity Beginner Course
Link
See Plans
Get AI Credits
LEARNETH
Tutorials list
Syllabus
3. Primitive Data Types
3 / 19
All these data types have default values, as shown in the contract (line 29).
You can learn more about these data types as well as Fixed Point Numbers, Byte Arrays, Strings, and more in the Solidity documentation.
Later in the course, we will look at data structures like Mappings, Arrays, Enums, and Structs.
Watch a video tutorial on Primitive Data Types.
Assignment
1. Create a new variable newAddr that is a public address and give it a value that is not the same as the available variable addr.
2. Create a public variable called neg that is a negative number, decide upon the type.
3. Create a new variable, newU that has the smallest uint size type and the smallest uint value and is public.
Tip: Look at the other address in the contract or search the internet for an Ethereum address.
Check Answer
Show answer
Next
Well done! No errors.
Compile
Home
introduction.sol
basicSyntax.sol
primitiveDataTypes.sol
Create a DApp with your contracts
Start now
22 */
23 int8 public i8 = -1;
24 int public i256 = 456;
25 int public i = -123; // int is same as int256
26
27 address public addr = 0xCA35b7d915458EF540aDe6068dFe2F44E8fa733c;
28
29 // Default values
30 // Unassigned variables have a default value
31 bool public defaultBoo; // false
32 uint public defaultUint; // 0
33 int public defaultInt; // 0
34 address public defaultAddr; // 0x0000000000000000000000000000000000000000
35
36 // New values
37 address public newAddr = 0x0000000000000000000000000000000000000000;
38 int public neg = -12;
39 uint8 public newU = 0;
40 }
Explain contract
AI copilot
Filter with transaction hash or address
[vm] from: 0x5B3...eddC4 to: Counter.(constructor) value: 0 wei data: 0x608...20033 logs: 0 hash: 0xc1b...b5b21
[call] from: 0x5B38Da6a701c568545dCfcB03FcB875f56beddC4 to: Counter.get() data: 0x6d4...ce63c
[vm] from: 0x5B3...eddC4 to: Counter.inc() 0xd91...39138 value: 0 wei data: 0x371...303c0 logs: 0 hash: 0x3ec...ee5b5
[vm] from: 0x5B3...eddC4 to: Counter.dec() 0xd91...39138 value: 0 wei data: 0xb3b...cfa82 logs: 0 hash: 0xccc...79d46
Debug
Scam Alert
Initialize as git repo
Did you know? To prototype on a uniswap v4 hooks, you can create a Multi Sig Swap Hook workspace. Template created by the cookbook team.
RemixAI Copilot (enabled)

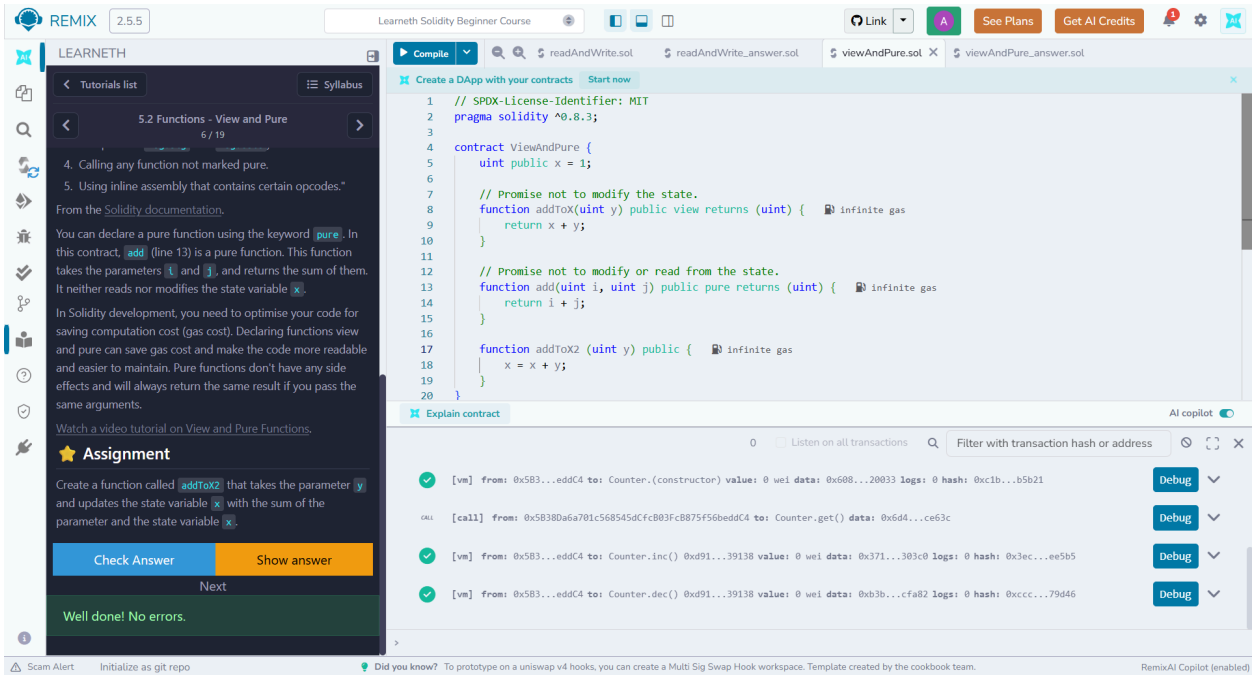
Tutorial 4


REMIX 2.5.5
Learneth Solidity Beginner Course
Link
See Plans
Get AI Credits
LEARNETH
Tutorials list
Syllabus
4. Variables
4 / 19
Global variables, also called Special Variables, exist in the global namespace. They don't need to be declared but can be accessed from within your contract. Global Variables are used to retrieve information about the blockchain, particular addresses, contracts, and transactions.
In this example, we use block.timestamp (line 14) to get a Unix timestamp of when the current block was generated and msg.sender (line 15) to get the caller of the contract function's address.
A list of all Global Variables is available in the Solidity documentation.
Watch video tutorials on State Variables, Local Variables, and Global Variables.
Assignment
1. Create a new public state variable called blockNumber.
2. Inside the function doSomething(), assign the value of the current block number to the state variable blockNumber.
Tip: Look into the global variables section of the Solidity documentation to find out how to read the current block number.
Check Answer
Show answer
Next
Well done! No errors.
Compile
Home
variables.sol 3
variables_answer.sol 3
Create a DApp with your contracts
Start now
1 // SPDX-License-Identifier: MIT
2 pragma solidity ^0.8.3;
3
4 contract Variables {
5 // State variables are stored on the blockchain.
6 string public text = "Hello";
7 uint public num = 123;
8 uint public blockNumber;
9
10 function doSomething() public { 22334 gas
11 // Local variables are not saved to the blockchain.
12 uint i = 456;
13
14 // Here are some global variables
15 uint timestamp = block.timestamp; // Current block timestamp
16 address sender = msg.sender; // address of the caller
17 blockNumber = block.number;
18 }
19 }
Explain contract
AI copilot
Filter with transaction hash or address
[vm] from: 0x5B3...eddC4 to: Counter.(constructor) value: 0 wei data: 0x608...20033 logs: 0 hash: 0xc1b...b5b21
[call] from: 0x5B38Da6a701c568545dCfcB03FcB875f56beddC4 to: Counter.get() data: 0x6d4...ce63c
[vm] from: 0x5B3...eddC4 to: Counter.inc() 0xd91...39138 value: 0 wei data: 0x371...303c0 logs: 0 hash: 0x3ec...ee5b5
[vm] from: 0x5B3...eddC4 to: Counter.dec() 0xd91...39138 value: 0 wei data: 0xb3b...cfa82 logs: 0 hash: 0xccc...79d46
Debug
Scam Alert
Initialize as git repo
Did you know? To prototype on a uniswap v4 hooks, you can create a Multi Sig Swap Hook workspace. Template created by the cookbook team.
RemixAI Copilot (enabled)

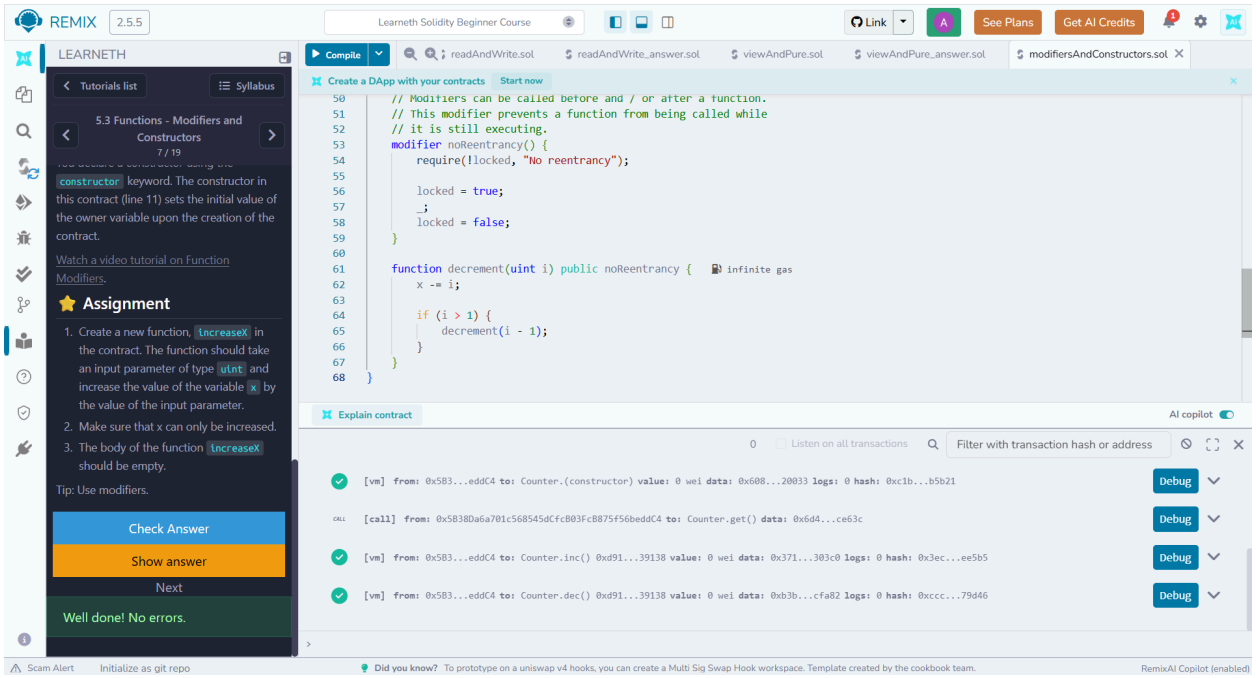
Tutorial 5



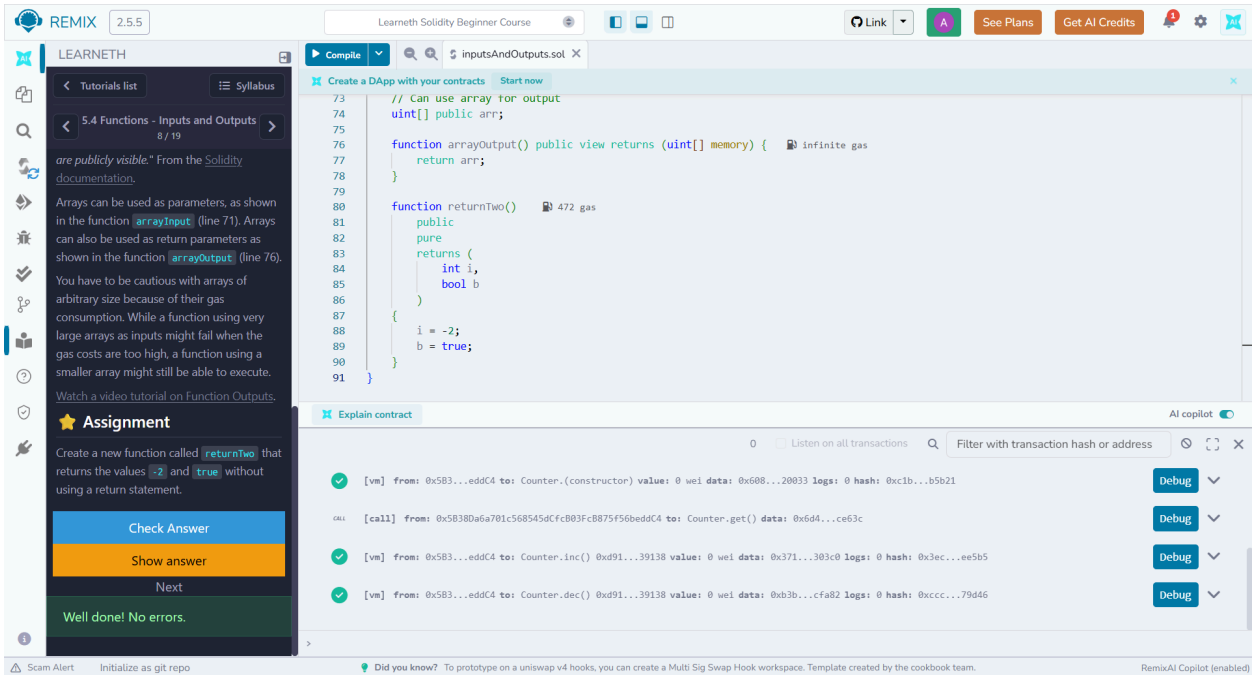
Tutorial 6



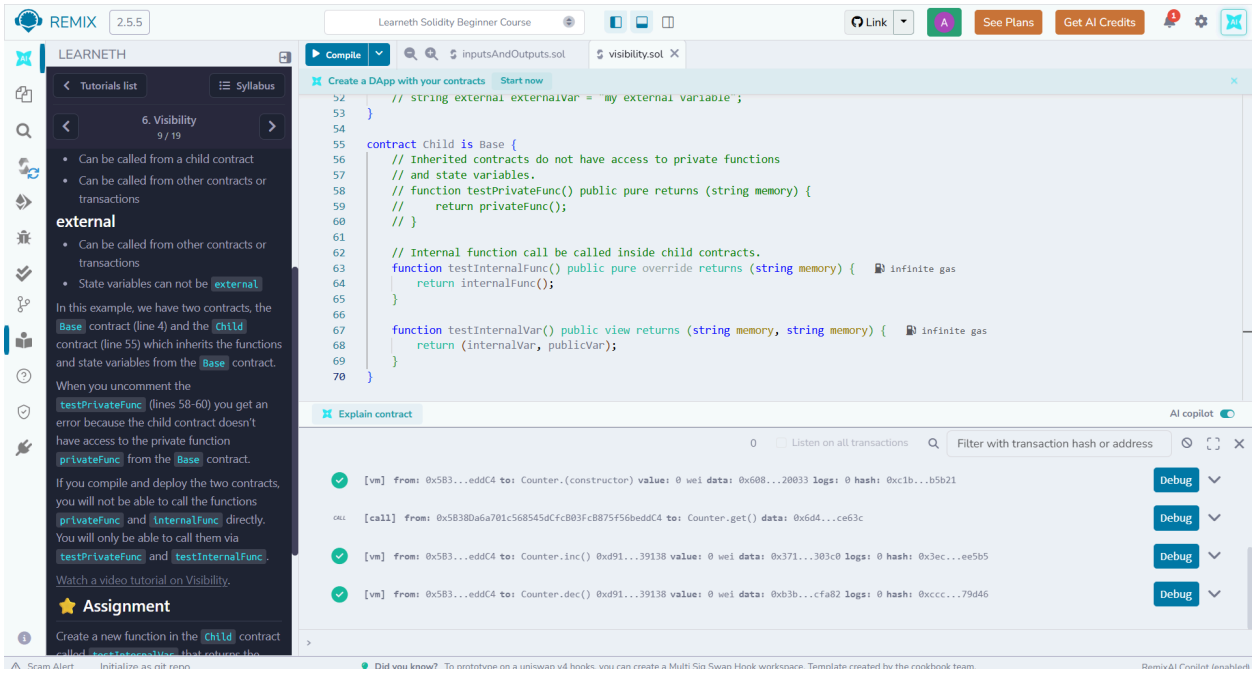
Tutorial 7



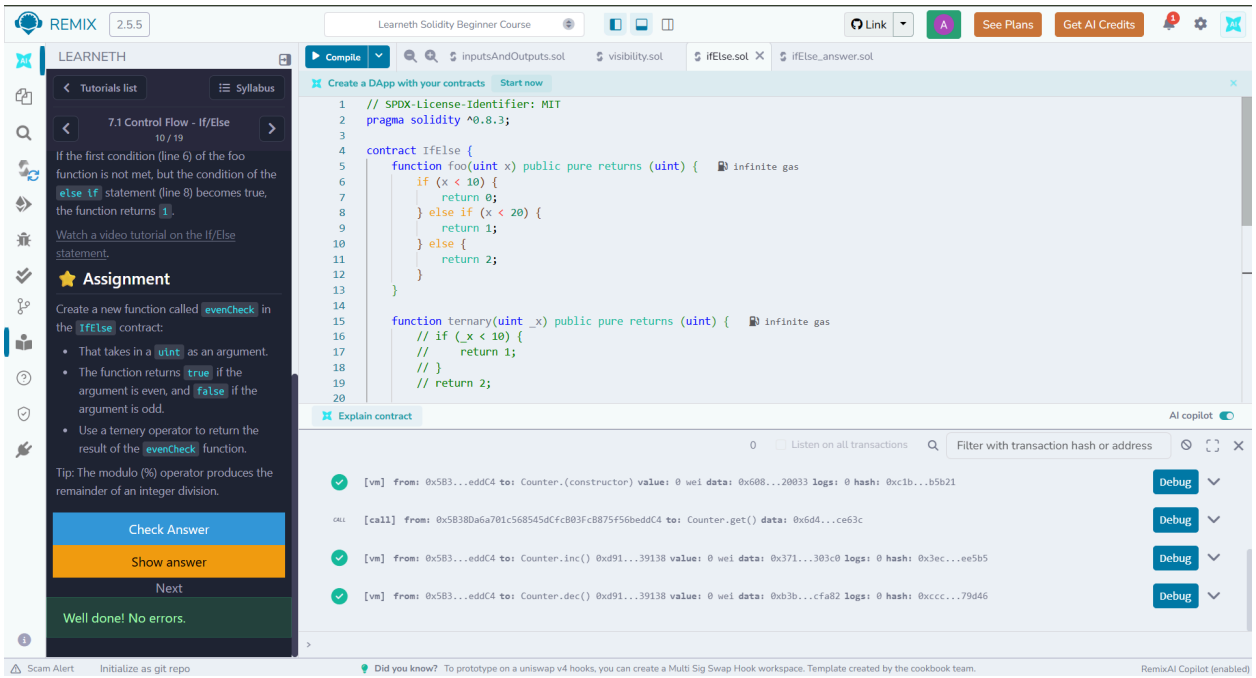
Tutorial 8



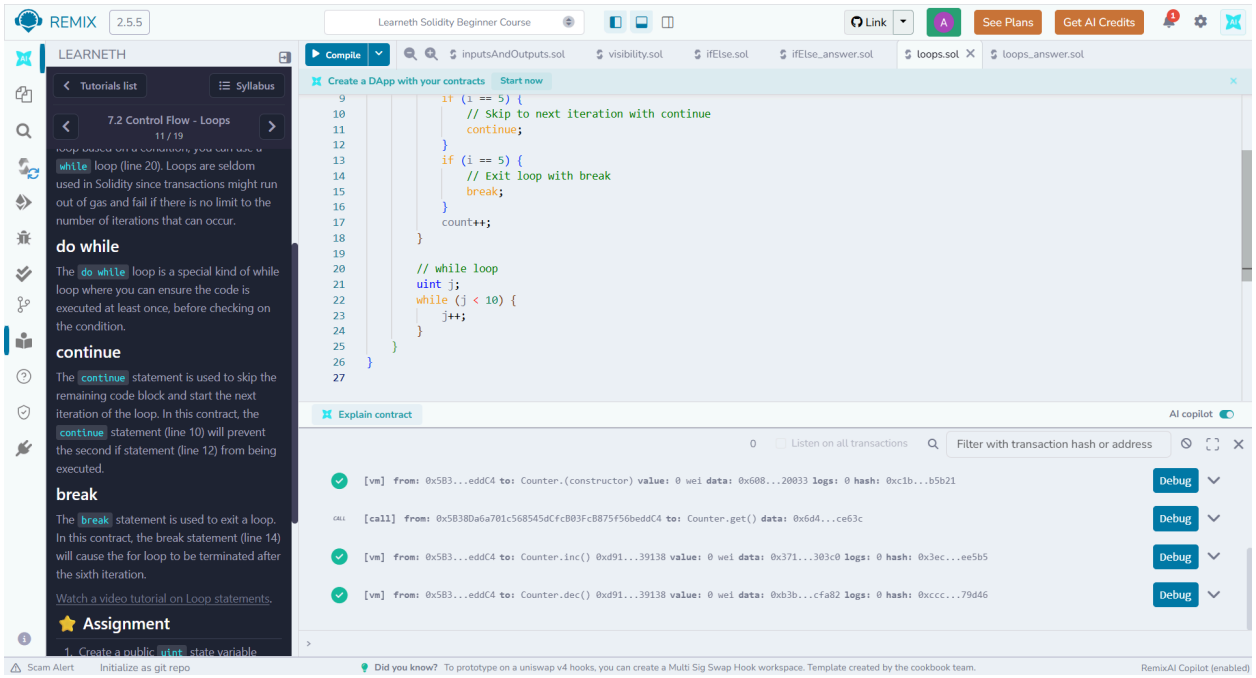
Tutorial 9



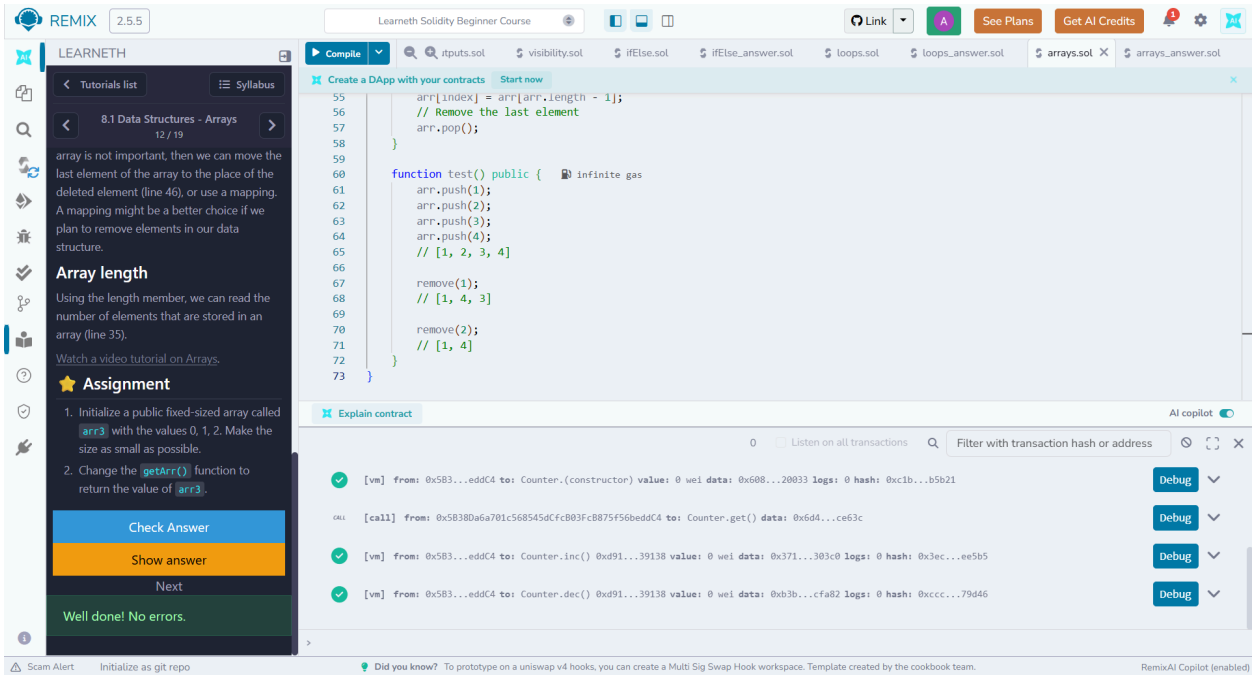
Tutorial 10



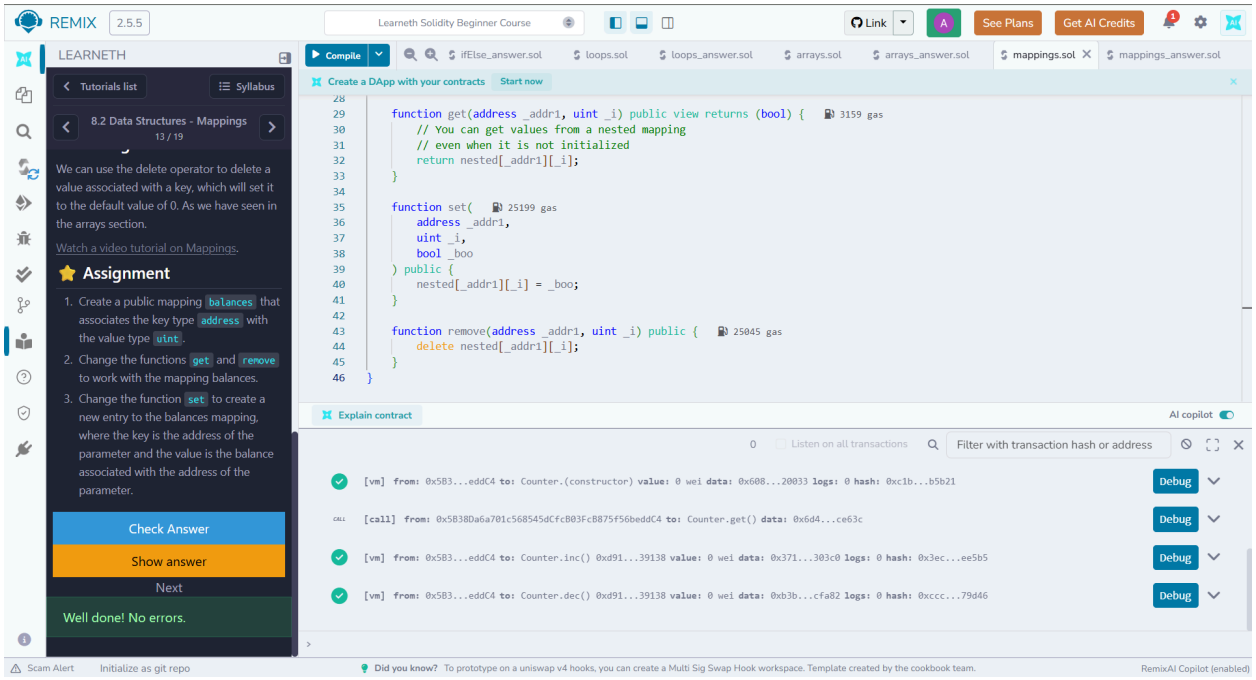
Tutorial 11



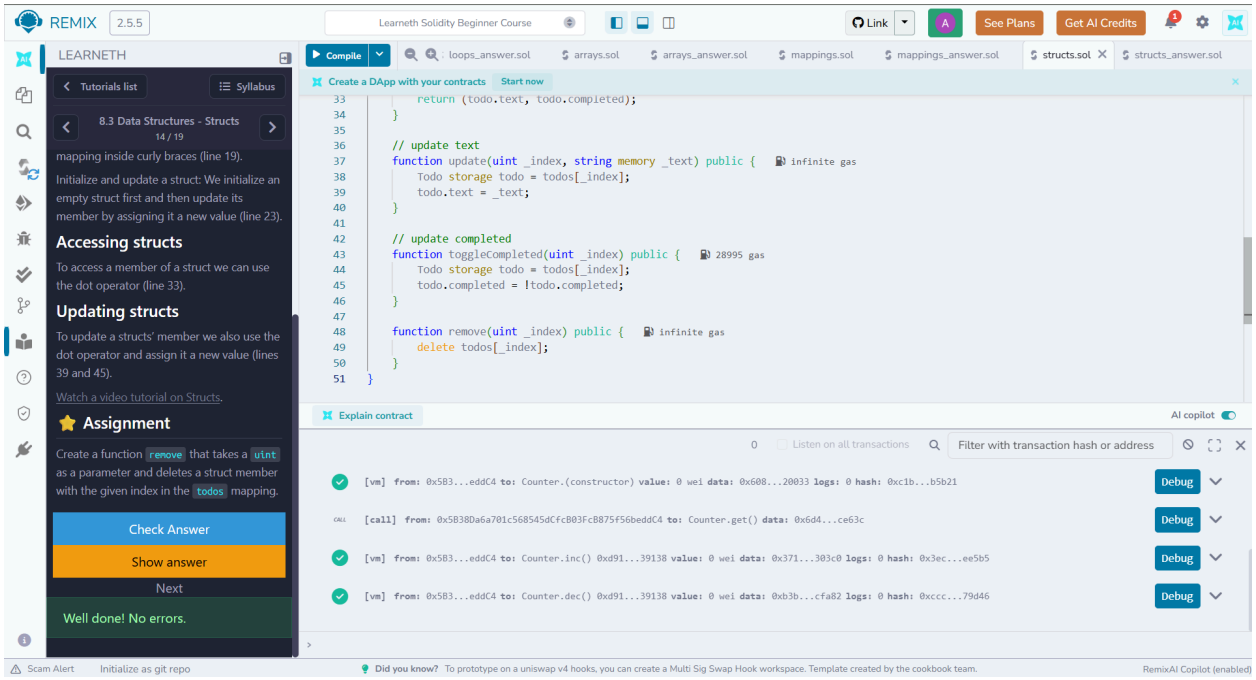
Tutorial 12



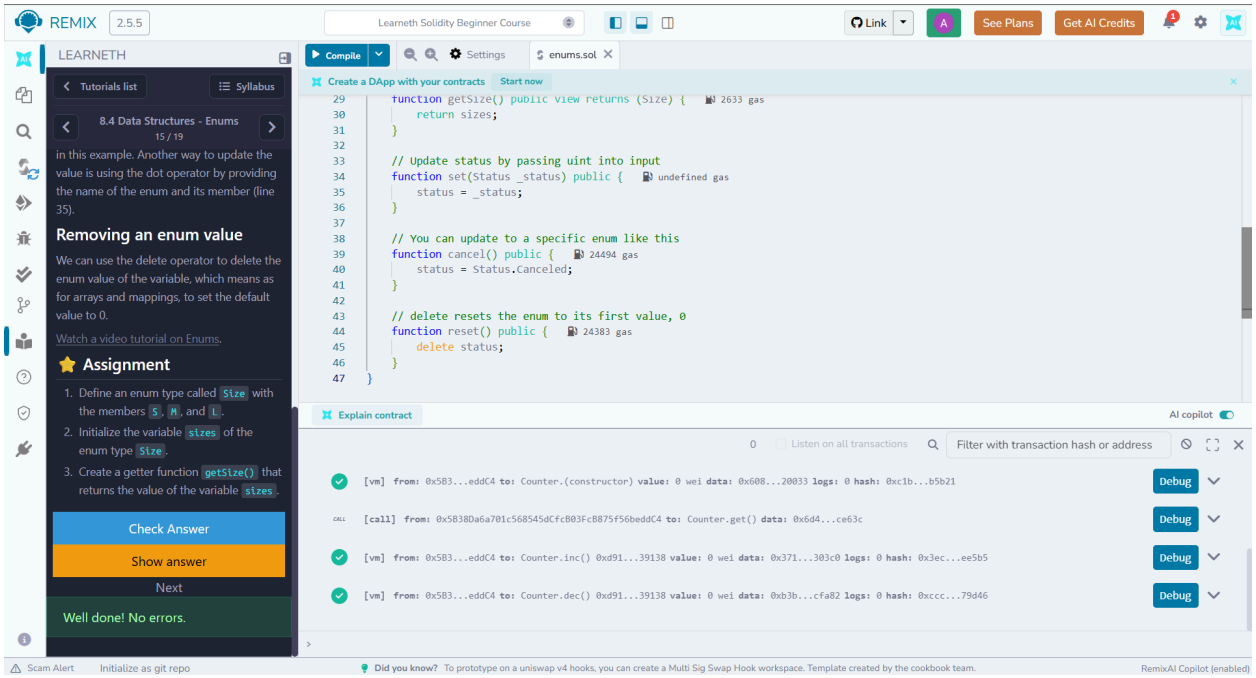
Tutorial 13



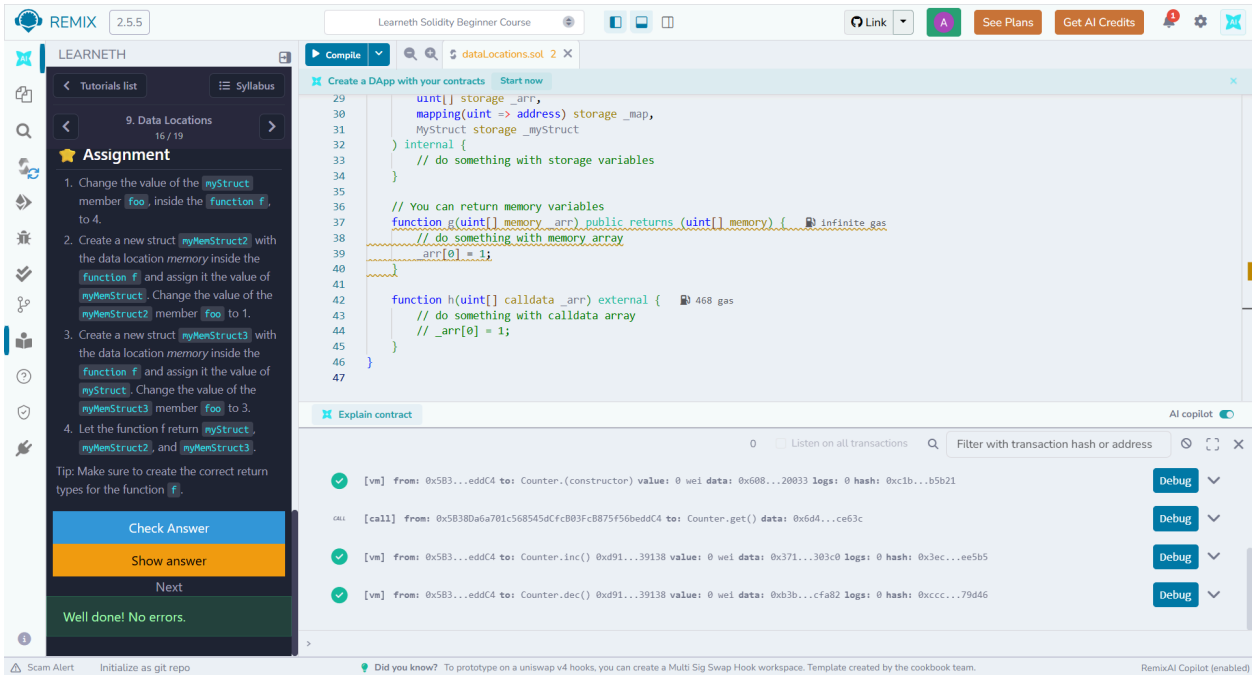
Tutorial 14



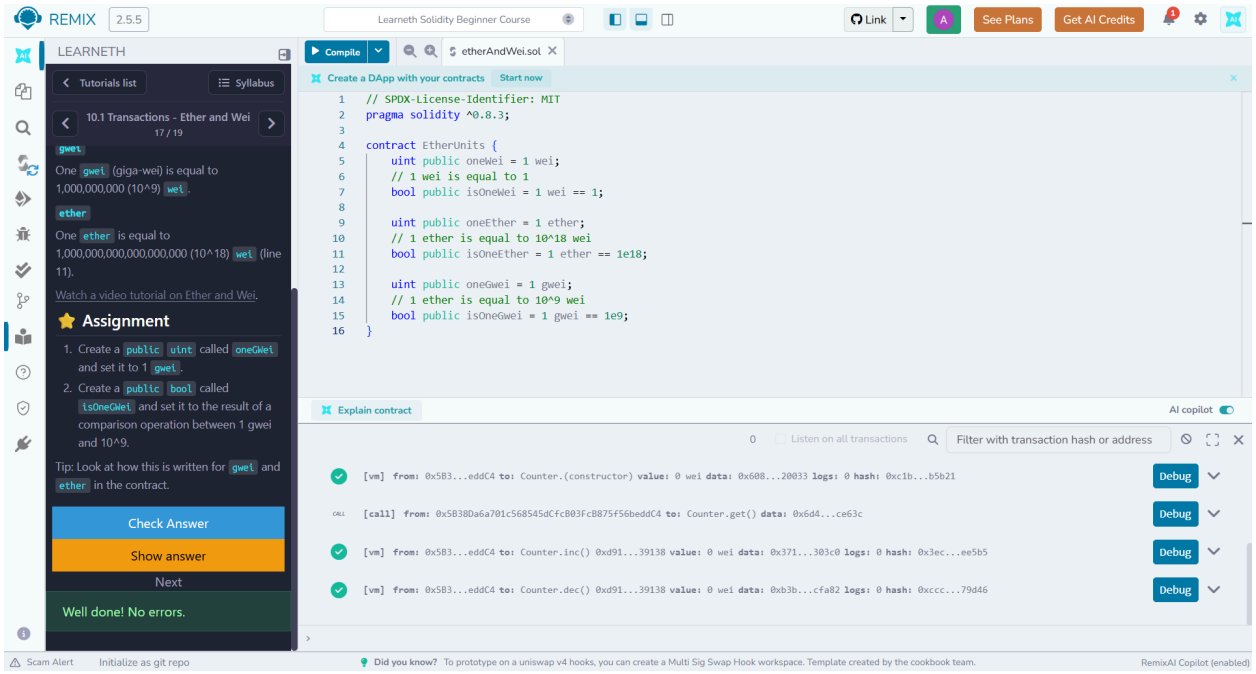
Tutorial 15



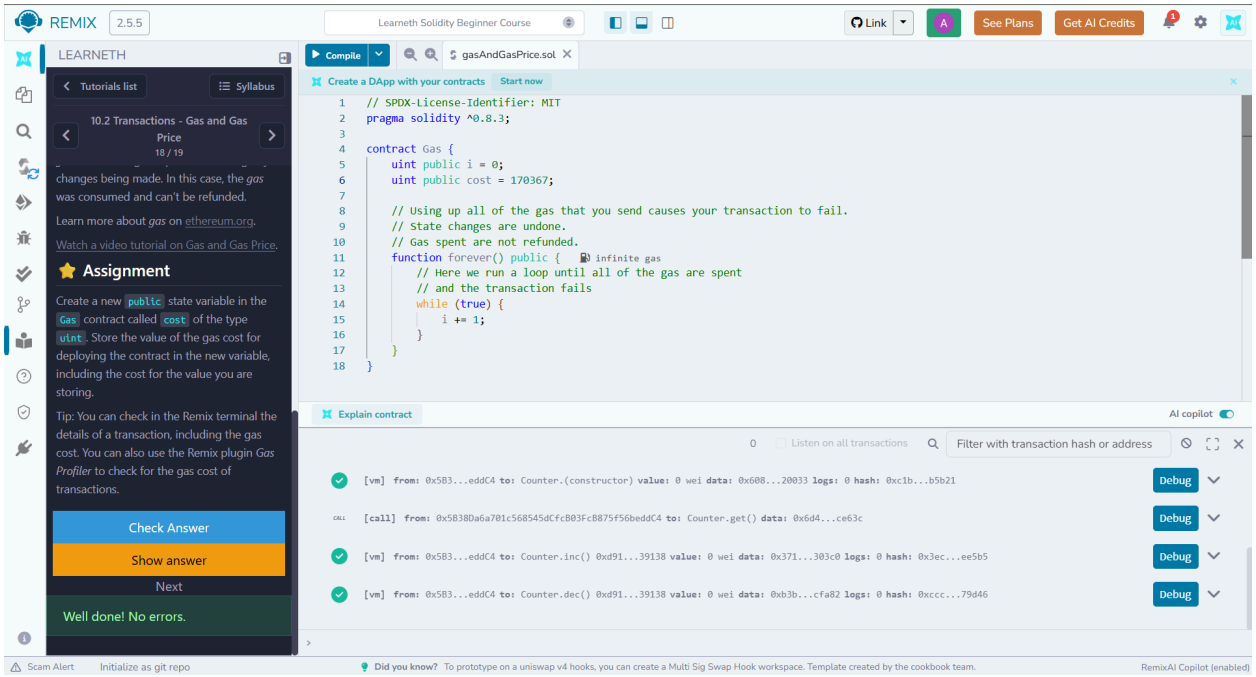
Tutorial 16



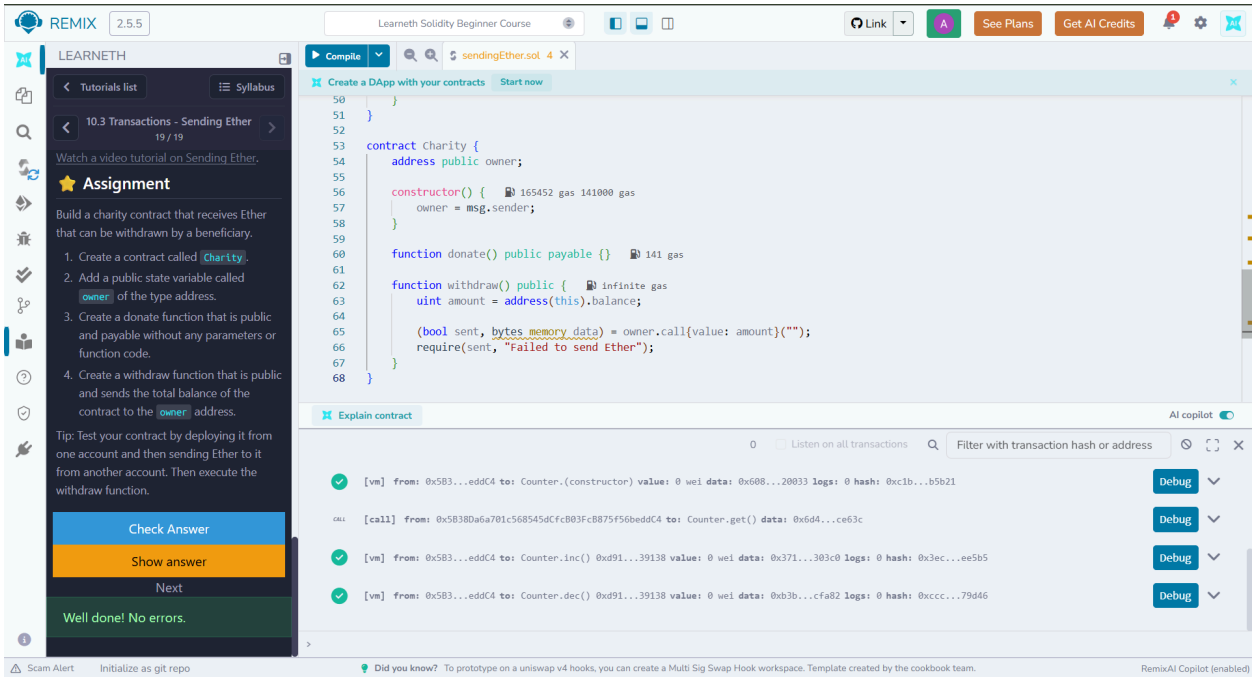
Tutorial 17



Tutorial 18



Tutorial 19



Conclusion: Through this experiment, the basics of Solidity programming and smart contract development were successfully learned using the Remix IDE. Key concepts like state variables, functions, modifiers, and contract deployment were practically implemented and tested on the Remix VM. Overall, this experiment provided valuable hands-on experience in writing, compiling, and executing smart contracts on the blockchain.